

System Design: ชีพจรหุ้น (Stock Sentiment Pipeline)

โปรเจกต์ — Data Engineering / AI Engineer

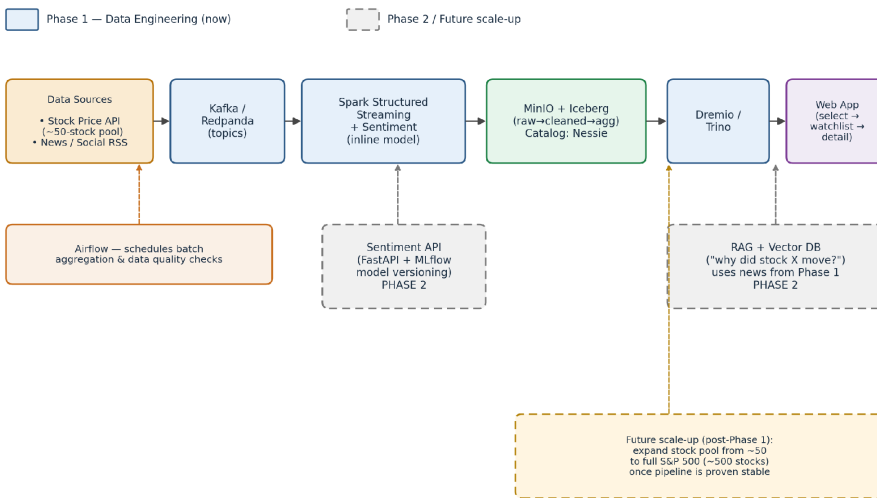
1. เป้าหมายโปรเจกต์

สร้างระบบ Data Pipeline ที่รับข้อมูลราคาหุ้นและข่าว/โซเชียลแบบ real-time ประมวลผล sentiment วิเคราะห์ความสัมพันธ์กับราคาหุ้น จัดเก็บบน Data Lakehouse (MinIO + Apache Iceberg + Nessie) และแสดงผลผ่าน web app ที่ผู้ใช้เลือกหุ้นเข้า watchlist เองได้ โปรเจกต์แบ่งเป็น 2 เฟส: เฟส 1 เน้น Data Engineering (ทำในปีนี้) และเฟส 2 เน้น AI Engineer (เริ่มต้นปีหน้า)

2. Timeline

Phase	Period	Focus
Phase 1	15 Sep – 30 Nov	Data Engineering
Phase 2	1 Jan – April	AI Engineer

3. Data Flow Diagram



4. Stock Scope

เฟส 1 เริ่มต้นด้วย pool หุ้นประมาณ 50 ตัว (คัดจากหลาย sector เช่น Technology, Automotive, Energy, Aerospace) เพื่อควบคุมปริมาณ API call และ storage ให้เหมาะกับเวลา 11 สัปดาห์ หากเฟส 1 สำเร็จและ pipeline เสถียร จะขยาย pool เป็นทั้ง S&P 500 (~500 ตัว) ในระยะถัดไป โดยไม่ต้องปรับสถาปัตยกรรมใหม่ เพราะออกแบบ schema ให้รองรับการเพิ่มหุ้นได้ตั้งแต่แรก

5. Web App Flow (3 หน้าหลัก)

ลำดับ	หน้า	หน้าที่
1	Stock Selection	ผู้ใช้เลือกหุ้นจาก pool ~50 ตัว (filter ตาม sector, ค้นหาได้) เพื่อเพิ่มเข้า watchlist ส่วนตัว
2	Watchlist Overview	แสดงเฉพาะหุ้นที่เลือกไว้เป็นการจัด: ราคา, % เปลี่ยนแปลง, sentiment badge, กราฟย่อ
3	Stock Detail Dashboard	คลิกจากการ์ดใน watchlist เข้าสู่กราฟราคา/sentiment แบบละเอียด, live news feed, system health

6. Component Breakdown

Layer	Tool	Purpose
Ingestion	Kafka / Redpanda	รับ stream ราคาหุ้น (~50 ตัว) + ข่าว แยก topic
Processing	Spark Structured Streaming	Clean/transform + join sentiment กับราคา (windowed join)
ML	LSTM / FinBERT	ให้คะแนน sentiment จากข้อความข่าว
Storage	MinIO + Apache Iceberg	Object storage + table format (raw → cleaned → aggregated)
Catalog	Nessie	Version control ของ Iceberg table (branch dev/prod)
Orchestration	Airflow	Schedule batch aggregation + data quality checks
Query	Dremio / Trino	SQL query ข้าม storage layer
Web App	Streamlit / Grafana	Stock selection, watchlist, และ dashboard รายละเอียด

7. Key Design Decisions

- ใช้ Apache Iceberg (แทน Delta Lake/Hudi) เพราะ engine-agnostic, รองรับ time travel และ schema evolution
- แยก raw / cleaned / aggregated layer เพื่อ traceability และ debug ง่าย
- Nessie catalog ให้ทำ branch แยก dev/prod โดยไม่กระทบข้อมูล production
- จำกัด pool หุ้นไว้ที่ ~50 ตัวในเฟส 1 เพื่อควบคุม scope ให้เหมาะกับเวลา และออกแบบ schema ให้ขยายเป็น S&P 500 ได้ในอนาคตโดยไม่ต้องปรับ architecture ใหม่
- แยกขั้นตอนเลือกหุ้น (selection) ออกจากหน้า dashboard เพื่อให้ผู้ใช้ควบคุมสิ่งที่อยากติดตามได้เอง

8. Non-functional Requirements (เฟส 1)

Idempotency (ป้องกันข้อมูลซ้ำด้วย Iceberg merge/upsert) • Fault tolerance (Kafka consumer group + Spark checkpointing) • Data quality (ตรวจสอบ schema/null/ค่าผิดปกติก่อนเข้า cleaned layer) • Scalability (schema รองรับขยายจาก ~50 เป็น ~500 ตัวได้โดยไม่ต้อง re-architecture)